

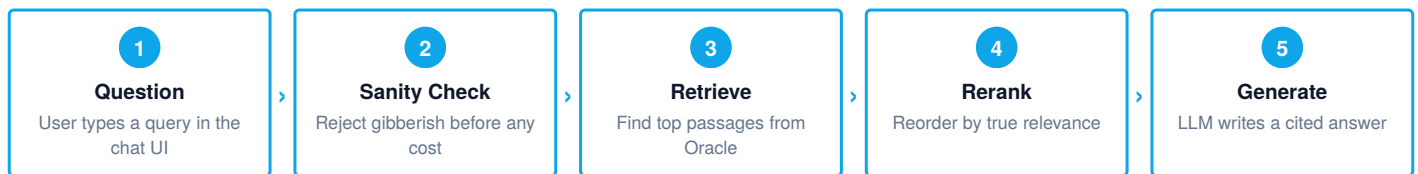
AudioLab AI

AUDIO RESEARCH COMPANION — FULL SYSTEM PIPELINE

A complete reference: what we built, why we built it, and how to improve it · Compiled May 2026

AudioLab AI is a local research assistant that answers technical questions about audio signal-processing papers. You ask a question; it retrieves the most relevant passages from a private library of 620 paper-chunks, then a language model writes a grounded, cited answer. This document walks the whole pipeline end to end.

The Pipeline at a Glance



Why This Architecture

It is grounded, not guessing

A plain chatbot invents facts. AudioLab answers **only from retrieved passages**, each labelled [1], [2] ... so every claim is traceable to a source. This is the core idea of **Retrieval-Augmented Generation (RAG)**.

It runs on your machine

The paper library and database live locally (Oracle 23ai). Your documents never leave your computer. The only thing that can go out is the question text — and only if you choose the paid cloud model.

Retrieval Quality — Measured, Not Assumed

Before trusting the system, we measured retrieval against a hand-built test set. These scores are saturated (essentially perfect on the current library):

1.000 MRR (RANK OF FIRST HIT)	1.000 HIT @ 5	0.995+ NDCG @ 5	620 INDEXED CHUNKS	22s FULL RE-EMBED TIME
---	-------------------------	---------------------------	------------------------------	----------------------------------

Why it matters: a perfect MRR means the single most relevant passage is almost always ranked first — the LLM gets the best evidence at the top, every time.

The Five Stages — What & Why

1 · The Chat Interface BUILT

What: A web UI (Streamlit) on `localhost:8502` with a conversation list, retrieval controls, and a provider switcher.

Why Streamlit: it turns Python into a web app with no separate front-end code — fast to build and maintain for a solo engineer. Trade-off: limited custom styling, which is why the UI went through many polish passes.

2 · Query Sanity Check BUILT

What: Before spending any compute, a fast check rejects nonsense input ("buoh", "asdf") and asks the user to rephrase.

Why: it stops the system from hallucinating an answer to a meaningless question, and on the paid model it prevents wasting money on garbage queries. 21 of 21 test cases pass.

3 · Retrieval — Oracle 23ai + BGE Embeddings BUILT

What: Each paper is split into ~620 chunks. Each chunk is converted to a 768-dimension vector by the `bge-base-en-v1.5` model and stored in Oracle 23ai's native VECTOR type. A question becomes a vector too; Oracle finds the nearest chunks.

Why Oracle native VECTOR: keeps everything in one database the engineer already runs — no separate vector store to maintain. Why BGE: a strong open embedding model that runs locally on the RTX 3050 GPU, no API cost.

4 · Reranking — BGE Reranker BUILT

What: The first retrieval is fast but rough. A second model (`bge-reranker-v2-m3`) re-scores the top candidates by reading the question and each passage *together*, then reorders them.

Why two stages: fast vector search casts a wide net; the slower reranker gives a precise final order. This two-step "retrieve then rerank" is what pushes the metrics to near-perfect.

5 · Generation — Multi-Provider LLM READY

What: The top passages plus the question go to a language model, which writes the answer with citations. Two providers are wired in:

- **Ollama** (local, free) — runs `qwen2.5:7b` on your GPU. Zero cost, lower quality.
- **OpenAI** (cloud, paid) — `gpt-4o-mini` by default. Much stronger; needs account credit.

Why both: free local model for everyday use and privacy; paid cloud model for when answer quality really matters. A switch in the sidebar picks per session, with automatic fallback to Ollama if the cloud call fails.

Supporting Capabilities

Cross-session Memory BUILT

Conversations and facts persist in a local SQLite database, so the assistant remembers context across sessions. Exportable and importable.

Web & Code Tools BUILT

Optional web search (arXiv, Semantic Scholar) adds [\[w1\]](#) citations; a sandboxed code runner executes Python snippets for DSP simulations.

DSP Simulation Toolkit BUILT

26 verified signal-processing functions the model can call to compute real results rather than describe them.

Cost Tracking BUILT

Every paid call's tokens and dollar cost are logged to a separate SQLite file; today's spend shows live in the sidebar.

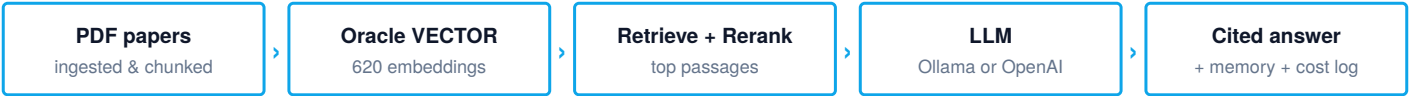
Technology Choices — Why Each One

LAYER	CHOICE	WHY THIS, NOT THE ALTERNATIVE
Vector store	Oracle 23ai native VECTOR	One database for everything; no extra service like Pinecone or a separate FAISS index to run and back up.
Embeddings	bge-base-en-v1.5 (768-dim)	Strong quality, runs locally on a 6 GB GPU, no per-call API fee. Larger models gave no measurable gain on this library.
Reranker	bge-reranker-v2-m3	Lifts precision of the final ordering; the single biggest quality win after basic retrieval.
UI	Streamlit 1.57	Pure-Python web app, no JavaScript build step. Trade-off: limited deep customisation.
Local LLM	Ollama + qwen2.5:7b	Free, private, offline. Ceiling: a 7B model struggles with complex tool use — hence the paid option.
Cloud LLM	OpenAI gpt-4o-mini	~\$0.001 per query, far stronger reasoning. Default model; user can switch to gpt-4o for hard questions.
Memory / cost DB	SQLite (two files)	Zero-setup, file-based, crash-isolated from each other. Perfect for single-user local use.

Honest Limitations

- **Local model ceiling.** The free 7B model cannot reliably use the DSP toolkit or follow complex instructions. Real power needs the paid model or a larger local one (14B+).
- **Retrieval is saturated.** On the current 620-chunk library, scores are already near-perfect — more retrieval tuning will not help. Growth must come from a *bigger library* or *new capabilities*.
- **UI ceiling.** Streamlit has been polished to its practical limit. Truly bespoke, app-like UI would need a separate multi-week front-end rewrite.
- **Sandbox is not airtight.** The code runner is defense-in-depth, not a hardened jail — fine for personal use, not for untrusted public input.

How the Pieces Connect



Each stage is independent — you can swap the LLM, grow the library, or add a tool without touching the others. That modularity is what lets the roadmap below proceed safely.

How to Improve — The Roadmap

The foundation is done. These are the next capability jumps, ordered by value and dependency.

Patent Search — EPO OPS NEXT

What: Search worldwide patents alongside papers, returning [P1] -style citations. **Why now:** the European Patent Office API is free and needs no language model, so it works today regardless of cloud-model billing. **How:** register free EPO credentials, add a backend module mirroring the web-search tool.

Image & Audio Upload — Vision + Whisper AFTER CREDITS

What: Drop in a spectrogram image or an audio clip; the system describes or transcribes it and folds it into the conversation. **Why:** turns a text tool into a true multi-modal audio assistant. **Needs:** OpenAI credit (vision + Whisper API).

Autonomous Agent — ReAct Loop AFTER CREDITS

What: Instead of one pass, the model plans, calls tools (retrieve, web, code, patents), observes results, and iterates until it has a complete answer. **Why:** the headline capability — it can solve multi-step research tasks on its own. **Needs:** a strong model (paid), because weak models loop or misuse tools.

Human-in-the-Loop Guardrails LATER

What: Approval gates before code runs or money is spent, plus an audit log of every model call. **Why:** makes the agent safe to trust with bigger tasks and gives you a spending safety net.

Three Concrete Ways to Get Better Answers

IMPROVEMENT	EFFORT	PAYOFF
Grow the library — ingest more papers	Low	Directly widens what the assistant can answer; retrieval is already excellent, so more content = more coverage.
Add OpenAI credit — switch default to gpt-4o-mini	5 minutes	Large jump in answer quality, reasoning, and reliable tool use. ~\$5 lasts thousands of queries.
Use gpt-4o for hard questions	One dropdown click	Best-in-class reasoning for the few queries that truly need it; everyday questions stay on the cheap model.

Bottom Line

AudioLab AI is a complete, measured, modular RAG system: **grounded answers, local-first privacy, perfect retrieval on its library, and a free/paid model switch.** The build phase is finished and verified. The path forward is not more fixing — it is **more content, stronger models, and new capabilities** (patents, multi-modal, agent), each of which slots cleanly into the existing pipeline.

AudioLab AI · Retrieval-Augmented Research Assistant for Audio Signal Processing · Pipeline Reference v1